

ORIGINAL RESEARCH

Dynamic equivalent modelling for active distributed network considering adjustable loads charging characteristics

Jingwen Wang | Jiehui Zheng  | Zhigang Li  | Qing-Hua Wu

School of Electric Power Engineering, South China University of Technology, Guangzhou, Guangdong, China

Correspondence

Jiehui Zheng, School of Electric Power Engineering, South China University of Technology, Guangzhou, Guangdong 510641, China.
Email: zhengjh@scut.edu.cn

Funding information

State Key Laboratory of Advanced Electromagnetic Technology, Grant/Award Number: AET 2024KF005; Guangdong Basic and Applied Basic Research Foundation, Grant/Award Number: 2023B1515250008; CSEE, Grant/Award Number: CSEE-YESS-2021014; National Natural Science Foundation of China, Grant/Award Number: 52477097

Abstract

As more renewable energy generators and adjustable loads such as electric vehicles are being connected to the power grids, load modelling of the distribution network becomes more complicated. Therefore, this paper explores a dynamic equivalent modelling method for active distribution network that takes into account electric vehicle charging. First of all the combination of integrated ZIP loads and motors is adopted as an equivalent model for active distribution networks. Subsequently, a four-layer, tri-stage deep reinforcement learning approach is used to solve the relevant key parameters of the proposed equivalent model. The method proposed in this paper fully utilizes the superiority of reinforcement learning in decision making, while the method combines the excellent feature extraction capability of deep learning. The method utilizes measurements obtained at boundary nodes to obtain an active distributed network equivalent model after a series of calculations. At the same time, adjustable loads are identified in detail. On the other hand, this method introduces a prioritized empirical playback mechanism, log-cosh loss function, and adaptive operator to improve the computational efficiency of the method. From the simulation results, the present method is effective.

1 | INTRODUCTION

With the growing environmental problems, countries around the world are increasing the percentage of renewable energy sources use for reducing pollution. The European Union (EU) countries, for example, have set targets to rise the share of new energy generators to over twenty percent by 2020 and more than twenty-seven percent by 2030 [1]. With the massive integration of renewable energy generators into power grids, smart grids have also seen unprecedented development. As a modern power grid, the smart grid enables the flexible flow of energy [2]. The future smart grid will operate under more complex conditions [3]. In this context, active distribution grids, as a key component of smart grids, are also facing the challenges posed by the integration of distributed renewable energy generators into power grids [4]. In addition, adjustable loads represented by electric vehicles (EVs) and energy storage devices are increasingly integrated with distributed energy sources (DERs), making the distribution grid more active and diverse. These conditions have led to the transformation of traditional distribution

grids towards active distribution networks (ADNs) [5, 6]. (The acronyms covered in this article will be presented in Table 1.)

At the same time, the distribution network equivalent modelling becomes more complex. Electric vehicles are an important part of the demand response regulation process of electric power system and play a positive role in the stable operation of it [7]. Ref. [8] proposes a multi-intelligent deep reinforcement learning method to derive charging schedules for electric vehicles in smart grids. In the smart grid fault scenario, ref. [9] focuses on the regulation capability of electric vehicles on the smart grid. At the same time, adjustable loads represented by EVs add uncertainty to the equivalent modelling study of ADNs. Accurate load modelling is one of the key factors affecting whether the power system can operate stably. This effect is especially pronounced in stochastic situations [10, 11]. Accurate load modelling describes the load flow within the ADNs well [12]. It is important to note that the load model needs to be stable relative to its external response.

Many scholars have conducted research in these related areas. In the operational control of active distribution grids, ref. [13]

This is an open access article under the terms of the [Creative Commons Attribution-NonCommercial-NoDerivs](https://creativecommons.org/licenses/by-nc-nd/4.0/) License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

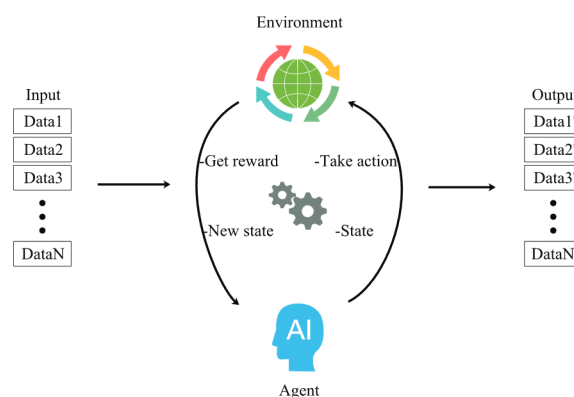
© 2024 The Author(s). *IET Generation, Transmission & Distribution* published by John Wiley & Sons Ltd on behalf of The Institution of Engineering and Technology.

TABLE 1 Acronyms table.

Acronym	Original words
ADN	Active distribution network
DER	Distributed energy sources
DRL	Deep reinforcement learning
EA	Evolutionary algorithm
EU	European union
EV	Electric vehicle
GZIP	General ZIP load
MAE	Mean absolute error
MDDQN	Modified DDQN
MOEA	Multi objective evaluation algorithm
MORL	Multi objective reinforcement learning
MSE	Mean square error
NLS	Non-linear least squares
PEC	Plug in electric cab
PSO	Particle Swarm algorithm
PV	Photovoltaic
RLS	Recursive least squares
SZIP	Special ZIP load
TD	Temporal difference
TVPI	Time varying parameter identification

proposes a new optimization strategy to optimize ADN by considering the siting and incentive price of responsive load with the objective of active management cost of source network loads. Based on an overview of the identification methods for a single operating point, when the operating conditions of the distribution network change, the equivalence quantity update problem is discussed in ref. [14]. In the modelling of active distribution networks, A comprehensive multi-voltage active distribution grid model based on actual grid data as well as load and generation time series is developed in ref. [15]. Ref. [16] discusses the equivalence quantity update problem when the operating conditions of the distribution network change. For the local stability problem of ADNs, a tuning method is proposed in ref. [17], and a suitable low-order ADN model is derived through an aggregation process.

Meanwhile, dynamic equivalence modelling of distribution networks has become one of the research directions of interest to many researchers in recent years. Equivalent induction motors are used as a dynamic model in work related to dynamic equivalent modelling [18]. Ref. [19] provides a metric to assess the accuracy and generalization capabilities of a few models in modern dynamic analysis of distribution networks. In ref. [20], an ADN-equivalent modelling approach is proposed that takes into account the spatial uncertainty of distributed renewable energy distribution locations. Ref. [21] suggests a three-stage strategy using field measurements to efficiently create a set of ADN dynamic models.

**FIGURE 1** Schematic of reinforcement learning.

Equivalence modelling process can be divided into two steps: determining the equivalence model structure and parameter identification. To obtain an accurate load model structure, it is necessary to use optimization methods for the identification of special parameters. Therefore, the load equivalent modelling problems can also be considered as a kind of optimization problems. Many mathematical methods are used to solve such problems. Ref. [22] presents a robust time-varying parameter identification (TVPI) method for integrated load modelling of distribution networks and solves it by using non-linear least squares (NLS). A robust recursive least squares (RLS) method is proposed in ref. [23] to estimate the time-varying parameters of the substation-level ZIP load model. Mathematical methods are widely used for their reliable solving ability and faster solving speed. However, the disadvantages of mathematical methods should not be ignored as well.

The mathematical approach to solving optimization problems has significant limitations when the initial value differs significantly from the actual value. Many heuristic algorithms are also applied to solve optimization problems. The robustness of heuristic algorithms represented by evolutionary algorithm (EA) [24] and particle swarm algorithm (PSO) [25] algorithms needs to be improved. When the environment is constantly changing, it is difficult for heuristic algorithms to achieve online recognition of parameters.

Reinforcement learning is a method of optimizing strategies based on the constant interaction of the agent with information about the state of the environment. It continuously modifies behavioural strategies with the goal of maximizing long-term rewards. The optimization process of reinforcement learning methods does not require an exact model of the optimization objective. Reinforcement learning also does not require too much a priori experience related to the state of the environment and is an online learning method that can be applied to real-time environments [26]. Therefore, reinforcement learning is more suitable for handling more complex ADN equivalence modelling problems.

The basic process of reinforcement learning can be represented in Figure 1. A multi-objective optimization algorithm called multi-objective reinforcement learning (MORL) is

proposed in ref. [27]. Simulation results show that MORL can find more accurate solutions faster compared to multi-objective evaluation algorithm (MOEA). In order to accurately characterize the charging load model of plug-in electric cabs (PECs), a multi-step q-learning method based on a multi-intelligent body framework was developed in ref. [28]. From the simulation results, this method can achieve dynamic modelling of the charging behaviour of PETs.

The computational power of reinforcement learning is excellent, and with the rapid development of deep neural networks, reinforcement learning has made significant progress [29]. Deep reinforcement learning (DRL), which combines the strengths of both, is ideally suited for solving more complex problems [30, 31]. The access of a large number of renewable energy generators and adjustable loads complicates the problem of equivalent modelling of ADNs. It makes sense to employ DRL to solve complex ADNs equivalent modelling problems. Deep reinforcement learning can utilize the computational ability of reinforcement learning to ensure the accuracy of the equivalent model, and at the same time utilize the powerful feature extraction capability of deep learning to track special load characteristics. However, there is not a lot of research related to modelling the equivalence of ADNs using deep reinforcement learning.

This paper explores the possibility of deep reinforcement learning to solve the ADNs equivalence modelling problem and propose a four-layer hierarchical reinforcement learning framework. Aiming at the complexity caused by the connection of a large number of renewable energy generating units and adjustable loads in the grid, we adopt a structure with ZIP loads connected in parallel with asynchronous motors as an equivalent model for ADNs. The special load characteristics are tracked while ensuring the accuracy of the ADNs equivalent modelling. We believe that this paper makes the following contributions:

- (1) Categorical identification of adjustable loads in an equivalent ZIP model to independently express the operating state of adjustable loads is possible.
- (2) A hierarchical reinforcement learning approach is proposed to ensure the accuracy of the model while recognizing adjustable load characteristics.
- (3) The classical DDQN algorithm is improved to ensure the accuracy of online parameter identification by using preferred empirical playback, log-cosh loss function, duelling networks, and hyper-parameter adaptive mechanism.

The main points of the rest of the paper are as follows: Firstly, the basic structure of the ADN is described in Section 2. The equivalent model for ADN is given in Section 3. Section 4 describes how to use the proposed four-layer hierarchical reinforcement learning for parameter identification after the basic structure of the equivalence model is determined. This part describes in detail the improvement ideas of the DDQN algorithm. The simulation arithmetic is given in Section 5 and the experimental results are analysed. Section 6 gives the conclusion.

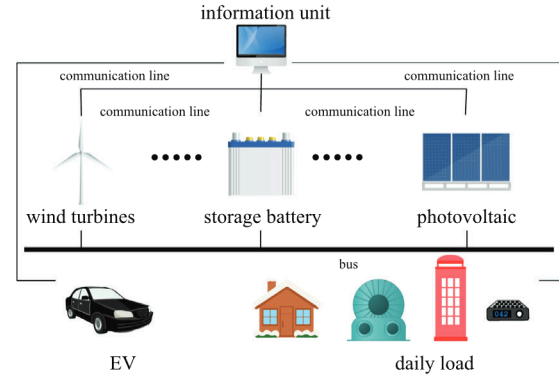


FIGURE 2 Active distribution network structure.

2 | ACTIVE DISTRIBUTION NETWORK STRUCTURE

The components of ADN studied in this paper include wind turbines, photovoltaic (PV) generators, energy storage devices and various types of loads. Its basic structure can be briefly represented schematically in Figure 2. Note that in this paper, loads are categorized into exceptional loads and daily loads. The energy storage device can be used as a load or as a generator, depending on the simulation scenario. Renewable energy generators convert other forms of power into electric power, which is an important physical quantity in equivalent modelling studies. This section describes the process of energy conversion and the resulting electric power is the basis for the work in Section 3.

2.1 | Mathematical modelling of wind turbines

In this study, we consider that wind turbines convert wind information into electrical information. The total amount of wind power generated at time t is equal to the sum of the power generated at time t by each wind turbine [32], which can be expressed as Equation (1).

$$P_{\text{total}}(t) = \sum_{i=1}^n P_i(t) \quad t \in [0, T] \quad (1)$$

where $P_i(t)$ denotes the power generated by a single wind turbine, $P_{\text{total}}(t)$ denotes the power generated by all wind turbines, and n denotes the number of wind turbines. Within the interval $[0, T]$, the maximum power moment of a single wind turbine can be denoted as t_i and the maximum power moment of all generators can be denoted as t_{total} . They can be expressed as Equation (2).

$$\begin{cases} t_i = \max_{t \in T} P_i(t) \\ t_{\text{total}} = \max_{t \in T} P_{\text{total}}(t) \end{cases} \quad (2)$$

Then the maximum power of a single wind turbine and the maximum power of all wind turbines can be expressed as Equation (3).

$$\begin{cases} P_{i,\max} = P_i(t_i) \\ P_{\text{total},\max} = \sum_{i=1}^m P_i(t_{\text{total}}) \end{cases} \quad (3)$$

where $P_{i,\max}$ denotes the maximum power of the single wind turbine, $P_{i,\max}$ denotes the maximum power of all wind turbines. The output power of a single wind turbine at time t is expressed as Equation (4).

$$P_i(t) = \frac{1}{2} \cdot A_{\text{ws}} \cdot V_{\text{wind}}^3 \cdot C_{\text{power}} \cdot D_{\rho} \cdot \eta_{\text{wind}} \quad (4)$$

where A_{ws} denotes the wind swept area, V_{wind} denotes the wind speed, C_{power} denotes the wind energy conversion rate value, D_{ρ} denotes the air density, and η_{wind} is a coefficient.

2.2 | Mathematical modelling of PV

We consider photovoltaic generators as equipments that converts solar irradiance information into electrical information. When the PV generators are running, the relationship between the current and voltage can be expressed as Equations (5) and (6) [33].

$$\begin{aligned} I = & -I_{\text{re}} \left[\exp \left(\frac{V_{\text{solar}} + IR_{\text{ps}}}{mV_{\text{th}}} \right) - 1 \right] \\ & - (V_{\text{solar}} + IR_{\text{ps}}) G_{\text{pc}} + I_{\text{solar}} \end{aligned} \quad (5)$$

where I_{solar} is the current from photo-generated current source, R_{ps} is a series parasitic resistance of photovoltaic circuit, G_{pc} is a parallel parasitic conductance of photovoltaic circuit, I is the terminal current, V_{solar} is the terminal voltage of photovoltaic circuit, I_{re} is the reverse saturation current, m is the number of photovoltaic panels in series, V_{th} is the thermal voltage.

$$\begin{aligned} V = & -I \left(R_{\text{ps}} + \frac{1}{G_{\text{pc}}} \right) + \frac{I_{\text{re}} + I_{\text{solar}}}{G_{\text{pc}}} \\ & - mV_{\text{th}} W \left[\frac{I_{\text{re}}}{nV_{\text{th}}G_{\text{pc}}} \exp \left(\frac{-I + I_{\text{re}} + I_{\text{solar}}}{mV_{\text{th}}G_{\text{pc}}} \right) \right] \end{aligned} \quad (6)$$

where W denotes the Lambert W function. Then the power generated by the PV unit can be expressed as Equation (7).

$$\begin{aligned} P_{\text{solar}} = & -\frac{V_{\text{solar}}^2 G_{\text{pc}} - V_{\text{solar}}(I_{\text{re}} + I_{\text{solar}})}{1 + R_{\text{ps}} G_{\text{pc}}} - \frac{V_{\text{solar}} m V_{\text{th}}}{R_{\text{ps}}} \\ & W \left\{ \frac{I_{\text{re}} R_{\text{ps}}}{mV_{\text{th}}(1 + R_{\text{ps}} G_{\text{pc}})} \exp \left[\frac{V + R_{\text{ps}}(I_{\text{re}} + I_{\text{solar}})}{mV_{\text{th}}(1 + R_{\text{ps}} G_{\text{pc}})} \right] \right\} \end{aligned} \quad (7)$$

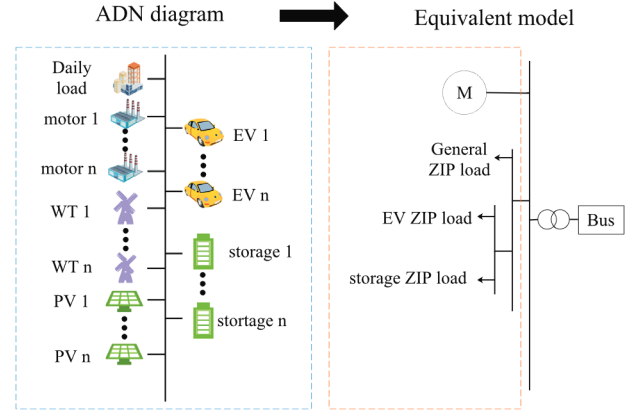


FIGURE 3 Equivalent model for ADN.

2.3 | Mathematical modelling of storage

The capacity of the energy storage device can be calculated according to Equation (8) [34].

$$L(t) = \int_t^{t+t'} P(\tau) d\tau \quad (8)$$

where $P(\tau)$ is the power load from storage device, $L(t)$ is the capacity of the energy storage device, t' is the termination time for sampling the energy storage device.

3 | EQUIVALENT MODEL OF ADN_s

As described in Section 2, the renewable energy generators convert other forms of information into electrical information, and the electrical information is the basis of the equivalent modelling. Based on the electrical information, the operation of each unit in the original ADN in Section 2 will be represented by the equivalent model in Section 3. In this paper, the power of the induction motor in the equivalent model corresponds to the power of the renewable energy generators. This paper adopts a comprehensive model combining ZIP loads [35] and induction motors as the equivalent model of ADN. The essence of ZIP load modelling is to use mathematical expressions to represent the correspondence between voltage and power. The generic model can be expressed as Equation (9).

$$\begin{cases} P_{Gi} = [a_{pGi}(V_{Gi})^2 + b_{pGi}(V_{Gi}) + c_{pGi}] \\ Q_{Gi} = [a_{qGi}(V_{Gi})^2 + b_{qGi}(V_{Gi}) + c_{qGi}] \end{cases} \quad (9)$$

where a_{pGi} , b_{pGi} , c_{pGi} denote active load factors respectively. a_{qGi} , b_{qGi} , c_{qGi} denote reactive load factors respectively. V_{Gi} denotes the voltage amplitude at node i . P_{Gi} denotes the active power of ZIP model and Q_{Gi} denotes the reactive power of ZIP model. This paper distinguishes the ZIP load into general ZIP load (GZIP) and special ZIP load (SZIP), and SZIP loads can be further subdivided and into EV loads and storage loads. The equivalent model for AND is shown in Figure 3.

3.1 | Equivalent load mathematical model of generators

This paper equates the generator to an induction motor that consumes negative power. Its mathematical model can be expressed as Equations (10) and (11).

$$\begin{bmatrix} E'_{d\text{-axis}} \\ E'_{q\text{-axis}} \end{bmatrix} = \begin{bmatrix} \frac{R_{\text{sta}} + X'X'}{\zeta} & \Delta \\ -\Delta & \frac{R_{\text{sta}} + X'X'}{\zeta} \end{bmatrix}^{-1} \begin{bmatrix} \frac{(X-X')(X'U_{d0} - R_{\text{sta}}U_{q0})}{\zeta} \\ \frac{(X-X')(R_{\text{sta}}U_{d0} + X'U_{q0})}{\zeta} \end{bmatrix} \quad (10)$$

Which $\Delta = \frac{-(X-X')R_{\text{sta}} + \zeta T'_{d0}(\omega-1)\omega_b}{\zeta}$, the synchronized rotating motor transient electromotive force can be described by a coordinate system with two parts, the D -axis component and the Q -axis component. $E'_{d\text{-axis}}$ is the d -axis component, and $E'_{q\text{-axis}}$ is the corresponding q -axis component. $X = X_l + X_e$, X_l is the stator leakage reactance and X_e is motor excitation reactance. X is the locked rotor short-circuit reactance, $X = X_l + \frac{X_e X_r}{X_e + X_r}$, X_r is the rotor leakage reactance. $\zeta = R_{\text{sta}}^2 + X'^2$, R_{sta} is the stator resistance. The motor terminal voltage has two parts, the D -axis component and the Q -axis component. $U_{d\text{-axis}}$ is the d -axis component, and $U_{q\text{-axis}}$ is the q -axis component of it. $\omega_b = 2\pi f$ is the synchronous angular speed of the motor, ω is the rotational angular speed of the equivalent motor. The rotor slip is expressed as $s = 1 - \omega$. T'_{d0} is the rotor time constant, which can be expressed as $T'_{d0} = \frac{X_m X_r}{R_r \omega_b}$, where R_r is the resistance of the rotor.

$$\begin{cases} I_{d\text{-axis}} = \frac{1}{R_{\text{sta}} + X'^2} [R_{\text{sta}}(U_{d\text{-axis}} - E'_{d\text{-axis}}) + X'(U_{q\text{-axis}} - E'_{q\text{-axis}})] \\ I_{q\text{-axis}} = \frac{1}{R_{\text{sta}} + X'^2} [R_{\text{sta}}(U_{q\text{-axis}} - E'_{q\text{-axis}}) - X'(U_{d\text{-axis}} - E'_{d\text{-axis}})] \\ P_m = U_{d\text{-axis}} I_{d\text{-axis}} + U_{q\text{-axis}} I_{q\text{-axis}} \\ Q_m = U_{q\text{-axis}} I_{d\text{-axis}} - U_{d\text{-axis}} I_{q\text{-axis}} \end{cases} \quad (11)$$

The motor stator current has two components, the D -axis component and the Q -axis component. $I_{d\text{-axis}}$ is the d -axis component and $I_{q\text{-axis}}$ is the q -axis component. This paper introduces a ratio σ_{pm_i} to better describe the model and it can be expressed as Equation (12):

$$\sigma_{pm_i} = \frac{P_{moini}}{P_{ini}} = \frac{P_{absor_i} - P_{gene_i}}{P_{ini}} \quad (12)$$

where the total initial active power is P_{ini} , P_{moini} denotes the initial value of the active power for equivalent motor, P_{absor_i} denotes the absorbed power of induction motor, and the active

power of each generator is P_{gene_i} . the total initial active power can be described as Equation (13):

$$P_{ini} = P_{absor_i} + P_{gene_i} + P_{ZIP_i} \quad (13)$$

Then, when ZIP reaches the steady state, the load power is expressed as Equations (14) and (15),

$$P_{ZIP_i} = P_{ini} - P_{moini} \quad (14)$$

$$Q_{ZIP_i} = Q_{ini} - Q_{moini} \quad (15)$$

where Q_{ini} is the total reactive power. At this point, the PQ component of the equivalent motor can be expressed as Equations (16) and (17).

$$P_{motor} = \sum_{i=1}^n W_i \times P_{motor_i} \quad (16)$$

$$Q_{motor} = \sum_{i=1}^n W_i \times Q_{motor_i} \quad (17)$$

The PQ component of the ZIP load can be expressed as Equations (18) and (19):

$$P_{ZIP} = \sum_{i=1}^n W_i \times P_{ZIP_i} \quad (18)$$

$$Q_{ZIP} = \sum_{i=1}^n W_i \times Q_{ZIP_i} \quad (19)$$

It should be noted uniformly that n in Equation (16)–(19) denotes the total number of models. W_i is the weight of each load. Special emphasis, $\sum_{i=1}^n W_i = 1$. P_{ZIP} is the real power of the equivalent ZIP load and Q_{ZIP} is the reactive power of the equivalent ZIP load. The power of an equivalent motor consists of two components, and reactive power. P_{motor} is the active power and Q_{motor} is the reactive power.

3.2 | Equivalent load mathematical model of adjustable loads

Adjustable loads include EVs and energy storage devices in this paper. In fact, EV is a special kind of energy storage device. Therefore, electric vehicles and energy storage devices can be identified as a whole. In particular, in order to characterize adjustable loads in more detail, a categorical identification of them is also implemented herein. The operation of their can be regarded as a constant power operation process [36]. Therefore, in this paper, adjustable loads are equated to ZIP loads. Their operation can be described as Equation (20).

$$P_{adjustable} = P_{ZIP} \cdot \left[\alpha_p (V_t/V_0)^2 + \beta_p (V_t/V_0) + \gamma_p \right] \quad (20)$$

$\alpha_p, \beta_p, \gamma_p$ denotes special coefficient, and the sum of them is 1. V_0 is the initial voltage, V_t is the voltage at time t .

3.2.1 | Electric vehicles

The charging process of electric vehicles can be expressed as Equation (21).

$$\begin{cases} P_{EV} = W_{EV} \cdot P_{adjustable} \\ Q_{EV} = 0 \end{cases} \quad (21)$$

where W_{EV} is the weight of EV. There are two components to the power absorbed when charging an electric vehicle. P_{EV} is the active power of it, and Q_{EV} is the reactive power of it.

3.2.2 | Energy storage devices

The operation of an energy storage device can be expressed as Equation (22).

$$\begin{cases} P_{stor} = W_{stor} \cdot P_{adjustable} \\ Q_{stor} = 0 \end{cases} \quad (22)$$

where W_{stor} is the weight of EV, P_{stor} is the active power during energy storage devices operating, and Q_{stor} is the corresponding reactive power.

3.3 | Equivalent load mathematical model of daily load

The daily load can be expressed as Equation (23).

$$\begin{cases} P_{GZIP} = W_{GZIP} \cdot P_{adjustable} \\ Q_{GZIP} = Q_{ZIP} \end{cases} \quad (23)$$

GZIP denote the daily load. W_{GZIP} is the weight of GZIP, the sum of W_{GZIP} and $W_{stor-total}$ is 1. P_{GZIP} is the real power of GZIP, Q_{GZIP} is the reactive power of GZIP.

4 | METHOD OF SOLUTION

After determining the components of the equivalence model, a modified DDQN (MDDQN) is used to solve the key parameters of the model to ensure the accuracy of the equivalence model. The process is divided into three stages. The first stage roughly determines the parameters of the equivalent ADN model. The second stage accurately determines the parameters of the equivalent ZIP model and the equivalent motor on the whole by continuously adjusting the weights of the corresponding parameters on the basis of the previous stage. In the third stage, based on the previous stage, the parameters of adjustable loads in the equivalent model are identified internally. The active and reactive components of each load are finally obtained.

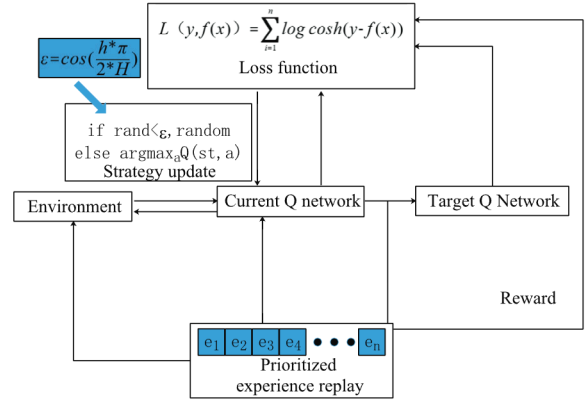


FIGURE 4 Brief flow of MDDQN.

4.1 | Algorithm

DQN is a deep reinforcement learning based on Q -values. It combines the powerful feature extraction capability of neural networks and the powerful computational capability of Q -learning. In order to avoid the problem of overestimation of Q -value by DQN, the DDQN algorithm is proposed in ref. [37]. The MDDQN proposed in this paper is an improvement on DDQN. A brief flow diagram of MDDQN is shown in Figure 4.

4.1.1 | Priority experience playback

Prioritizing experience playback is one of the excellent ways to improve sample utilization. In the original DDQN, the samples in the empirical playback pool were sampled uniformly. In this way, samples that are more effective for training are not utilized more efficiently. Samples with larger temporal-difference (TD) errors [38] are considered to be more useful for backpropagation and should be fully utilized. This paper ranks the samples in the empirical playback pool based on the size of the TD error, and samples with larger TD errors have a higher probability of being sampled. The TD error can be expressed as Equation (24).

$$\delta_{error} = \xi + \gamma \max_{d'} Q(\phi, d'; \theta_i^-) - Q(\phi, a; \theta_i) \quad (24)$$

where ξ is reward, γ is discount factor for computing temporal difference error, $Q(\phi, d'; \theta_i^-)$ denotes the Q -value at the next time, $Q(\phi, a; \theta_i)$ denotes the current Q -value.

4.1.2 | Log cosh loss function

The loss function of the original DDQN is the classical mean square error (MSE), which is continuously smooth at all points, easy to derive, and has a more stable solution. However, when the input values are far away from the true values, the corresponding loss values are large, which may lead to gradient explosion. Another classical loss function, the mean absolute error (MAE), has a stabilized gradient, which does not lead to the gradient explosion problem and has a more robust solution.

However, the centre point of the MSE function is collapsed and cannot be derived, which is very detrimental to convergence. This paper uses the log-cosh loss function. It is less sensitive to outliers, which makes it more robust in data where noise or outliers are present. In addition, the log-cosh loss is continuously derivable, which is very effective for deep neural network training such models that require gradient information. Finally, the log-cosh loss is a convex function, which means that the optimization problem is usually globally convergent and less likely to fall into a local optimum. It can be expressed as Equation (25).

$$L(y, \hat{y}) = \sum_{i=1}^n \log(\cosh(y_i - \hat{y}_i)), \cosh(x) = \frac{e^x + e^{-x}}{2} \quad (25)$$

where y_i is the true value and $\hat{y}_i$ is the predicted value.

4.1.3 | Adaptive ϵ -greed strategy

At the end of each learning round, the agent needs to follow a method to update the action strategy. ϵ -greedy is a commonly method used to update the action strategy for trade-off between exploration and exploitation. It can be expressed as Equation (26),

$$a = \begin{cases} \text{random,} & \text{if } rand < \epsilon \\ \arg \max_{a \in A} Q(s_t, a) & \text{others} \end{cases} \quad (26)$$

where $Q(s_t, a)$ is Q -value in DDQN, a is the action in reinforce learning. In the ϵ -greedy strategy, the agent has a certain probability to explore when choosing an action, that is, it chooses a random action in order to discover new states and strategies. In other cases, the agent chooses the best action based on the information it has learned, that is, it utilizes the existing knowledge. Specifically, the ϵ -greedy policy defines an exploration rate (exploration rate) ϵ , which usually takes values between 0 and 1, the value of it directly affects the randomization operation. If this random number is less than ϵ , a random action is chosen; if it is greater than or equal to ϵ , the optimal action is chosen based on the information that has been learned. In this paper, we adopt an adaptive strategy for the epsilon value, which can be expressed as Equation (27).

$$\epsilon = \cos\left(\frac{b * \pi}{2 * H}\right) \quad (27)$$

where, b is the current training count and H is the total training count. Agent need to explore unknown states and actions heavily in the early stages of learning and gradually reduce the rate of exploration and increase the possibility of utilizing what has been learned. This balances the need for exploration and exploitation and continuously optimizes the learning and convergence process of the strategy.

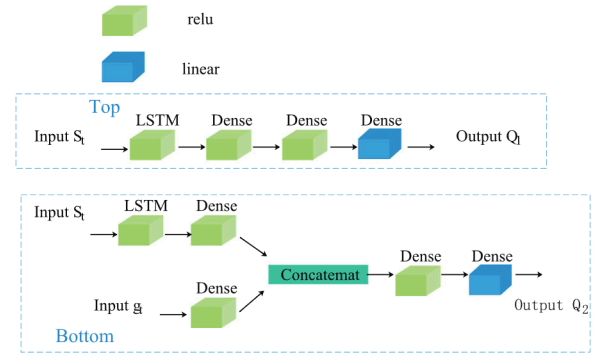


FIGURE 5 Neural network structure.

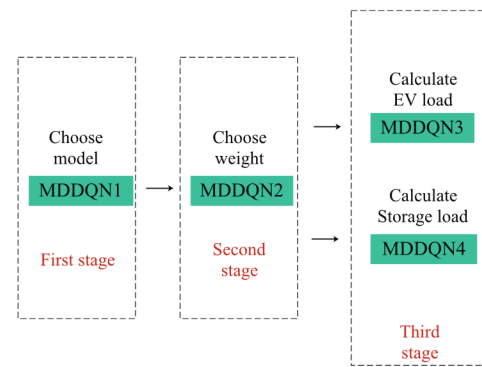


FIGURE 6 Solving procedure.

4.1.4 | Deep neural network structure

The deep neural network structure used in this paper is shown in Figure 5.

The input state S_t includes multidimensional data at multiple time points. The bottom layer of the deep neural network needs to obtain timely inputs, including the state S_t selected by the top layer and the target action g_t .

4.2 | Solving procedure

The equivalent modelling approach proposed in this paper is a three-stage approach. Its overall process is represented in Figure 6. Firstly, the first stage MDDQN selects the target g_t of the next stage under the state s_t . Next, the second stage selects the action a_t according to the target g_t selected by the first stage and its own current state s'_t , and the output will be used as the state of the next stage. Finally, the next stage selects the action a'_t according to the state.

First of all, in the first stage the desired model is roughly identified. And then, in the second stage the accurate PQ components of the equivalent model are obtained. Finally, in the third stage the PQ components of the particular load are internally identified.

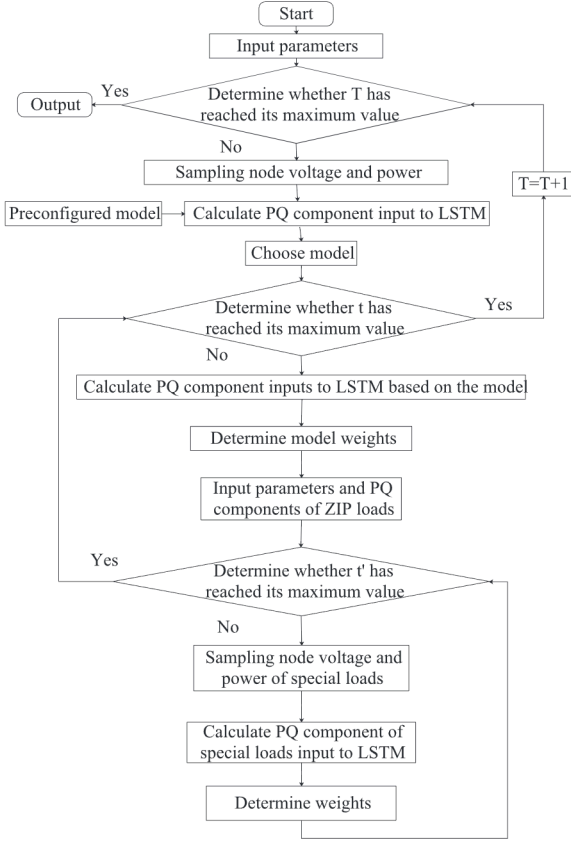


FIGURE 7 Flow chart of solving.

4.2.1 | Processes at each stage

The basic parameters of reinforcement learning need to be determined first, and then it is judged whether to output the results or not. If the simulation preset times are not reached, the data of voltage, power, and special load power are recorded every 15 min in a day cycle. Next, entering the first stage of reinforcement learning, each PQ component is computed based on a combination of predefined models. The predefined model values are fed into the LSTM neural network and the MDDQN algorithm is used to determine the combination of model parameters to be selected. This is followed by the second stage. The PQ components computed in the first stage are fed into the LSTM neural network in the second stage. The weights of each equivalent model were determined using MDDQN until the termination condition was reached. Based on the PQ components of ZIP loads calculated in the first two stages, the weights are adjusted in the third stage to determine the components of GZIP loads and adjustable loads, respectively. In this way, the internal identification of multiple types of adjustable loads is realized and the active and reactive power of the model is completely obtained. The process of the whole stage can be combined and represented as Figure 7.

4.2.2 | Algorithm design

In hierarchical deep reinforcement learning, the tasks are different between the layers. The task of the upper layer is to

maximize the overall reward and the task of the lower layer is to maximize its internal reward. In this paper, the reward function can be expressed as Equation (28).

$$Reward_t = 1/RE_t \quad (28)$$

where RE_t denotes the relative error between the calculated and measured values, that can be dealt with as Equation (29).

$$RE_t = 100 \times \frac{\sqrt{\frac{1}{N} \sum_{k=1}^N (y_{k\text{sam}} - y_{k\text{cal}})^2}}{\sqrt{\frac{1}{N} \sum_{k=1}^N y_{k\text{sam}}^2}} \quad (29)$$

where $y_{k\text{sam}}$ denotes the measured value, $y_{k\text{cal}}$ denotes the calculated value.

In this paper, the first stage determines the task of second stage g_t under the state S_t , and the second stage chooses the action a_t under the task g_t . The states of the first stage correspond to the PQ components calculated based on the predefined model, which is $S_t = [S_1, \dots, S_t, \dots, S_M]$, $S_i = [P_{\text{motor}}, Q_{\text{motor}}, P_{\text{ZIP}}, Q_{\text{ZIP}}]$. The action of the first stage of reinforcement learning can be represented as $g_t = [\text{model}_1, \dots, \text{model}_k]$, that is, selection from a predefined models. The size of the action space can be expressed as C_M^K . M denotes the total number of preset models and K denotes the number of preset models selected. The reward function for the first stage can be expressed as Equation (28).

$$Reward_1 = 1/RE_1 \quad (30)$$

In the second stage, state is the same as in the first stage. Action is the weight of each of the selected pre-determined models, which can be expressed as $a_t = [W_1, \dots, W_K]$. Then the reward of the second stage can be expressed as Equation 31 and 32.

$$Reward_2 = \begin{cases} 1, & \text{if } RE_2 < \mu \\ 0, & \text{otherwise.} \end{cases} \quad (31)$$

$$RE_2 = \beta_1 \times RE_{\text{motor}} + \beta_2 \times RE_{\text{ZIP}} \quad (32)$$

where $\beta_1 = 0.4$ and $\beta_2 = 0.6$ are coefficient, RE_{motor} and RE_{ZIP} are RE of motor and ZIP, respectively.

In the third stage, there are three layers of deep reinforcement learning in total. The state can be expressed as $S_{\text{thir}} = [P_{\text{ZIP}}, Q_{\text{ZIP}}]$, action is to determine their weights, which can be expressed as $a_{\text{thir}-1} = [W_{\text{EV}}, W_{\text{GZIP}}]$, $a_{\text{thir}-2} = [W_{\text{stor}}, W_{\text{GZIP}}]$, where $a_{\text{thir}-1}$, $a_{\text{thir}-2}$, $a_{\text{thir}-3}$ represent the action of each layer, respectively. the reward of the third stage can be expressed as Equations (33) and (34).

$$Reward_3 = \begin{cases} 1, & \text{if } RE_3 < \mu \\ 0, & \text{otherwise.} \end{cases} \quad (33)$$

$$RE_3 = \beta_3 \times RE_{\text{special}} + \beta_4 \times RE_{\text{GZIP}} \quad (34)$$

where $\beta_3 = 0.6$ and $\beta_4 = 0.4$ are coefficient, RE_{special} and RE_{GZIP} are RE of adjustable loads and GZIP, respectively, $RE_{\text{special}} = [RE_{\text{stor-total}}, RE_{\text{EV}}, RE_{\text{stor}}]$, where $RE_{\text{stor-total}}$,

TABLE 2 The preset parameters.

Item	M1	M2	M3	M4	M5
R_{sta}	0.001641	0.009907	0.007993	0.007993	0.000262
X_l	0.049191	0.074913	0.018014	0.018014	0.473784
X_e	3.452514	3.562868	3.360007	3.360007	3.327917
R_r	0.004872	0.015723	0.014504	0.014504	0.041983
X_r	0.028387	0.164548	0.179551	0.179551	0.482683
σ_{pm_i}	0.598436	0.631695	0.715900	0.572700	0.865538

TABLE 3 The preset parameters of loads.

Load	a_p	b_p	c_p	a_q	b_q	c_q
Storage-Device	0	0	1	0	0	0
EV	0	0	1	0	0	0
GZIP	0.252	0.612	0.136	0.533	-0.020	0.487

RE_{EV} , RE_{stor} represent the RE value of total-storage, EV and storage, respectively.

5 | CASE STUDY

In this paper, the effectiveness of the proposed MDDQN is verified, using PYTHON3.9 and tensorflow framework as a simulation tool. To verify the effectiveness of the proposed algorithm, the classical DQN and DDQN are used as the control group for the experiment.

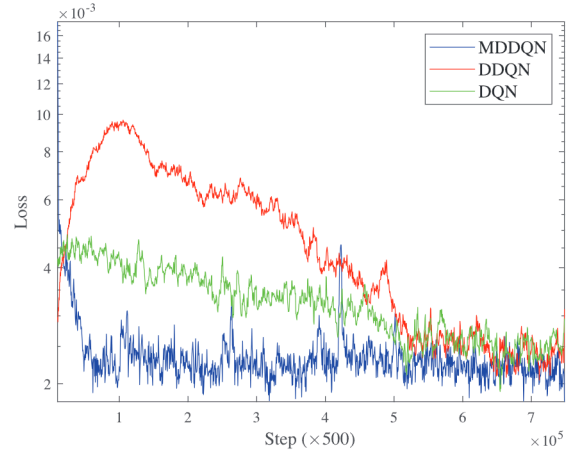
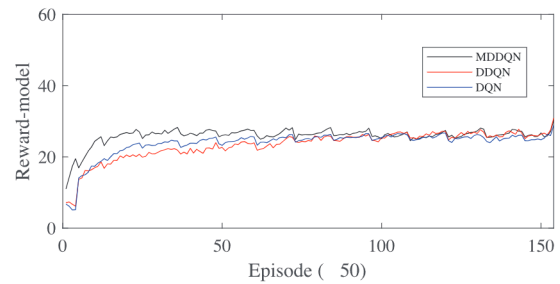
5.1 | Preset parameters

This paper uses IEEE14 node system [39] and IEEE33 node system [40] as transmission and distribution simulation examples for case study. Specifically, the IEEE14 node system is used as the transmission system, and the IEEE33 node system is connected at its 13th node as the distribution system. Step down transformers are installed at the junctions of transmission and distribution systems. The nodes in the distribution network where the PV units and wind turbines are located are nodes 7, 12, 16 and nodes 21, 25, 30 respectively. The remaining load nodes are evenly distributed with daily loads and adjustable loads. As mentioned earlier, this distribution system includes equivalent motors, GZIP loads, and various types of adjustable loads. Key information such as node voltage, active power, reactive power, and charging load ratio for adjustable loads are recorded in 15-min cycles. The preset information in this paper includes five sets of preset model parameters. These parameters are shown in Table 2.

The parameters of the load are shown in Table 3.

5.2 | Convergence analysis of MDDQN

The MDDQN used in this paper is obtained from the original DDQN. Therefore, the MDDQN has two networks, one is

**FIGURE 8** Loss function for MDDQN.**FIGURE 9** Reward for first stage.

current network and the other is target network. The current network selects an action, and this action is input to the target network to calculate the target value. The learning rate for each layer of MDDQN is 0.00025. The capacity of the storage unit in the first two stages is 5000 and the capacity of the storage unit in the third stage is 10,000. The second stage is based on the g_t chosen in the first stage for the selection of the action at. The loss function is shown in Figure 8.

It can be seen that all three algorithms converge. Specifically, the error of the DDQN algorithm rises from about 0.0028 to about 0.095 for the first 500 times of training, and then drops to convergence. The DQN algorithm gently decreases the training error from around 0.004 to convergence. The value of the loss function for MDDQN can be seen to decrease rapidly, basically fluctuating between 0.018 and 0.0029. This shows that the proposed method is effective and it ensures the convergence of the algorithm. Meanwhile, the MDDQN algorithm is trained with a large starting error, however, it converges at the fastest rate and shows the relative best performance.

5.3 | Validity analysis of algorithm

The level of reward value directly reflects the performance strength of the algorithm. As shown in Figures 9 and 10, they are the values of the reward function for the first and second stages, respectively.

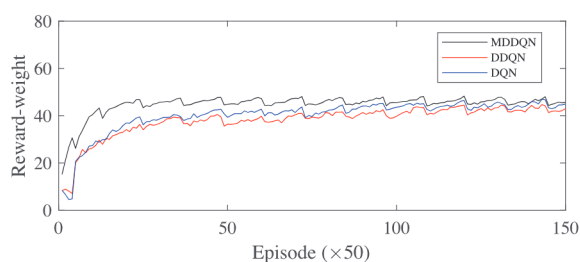


FIGURE 10 Reward for second stage.

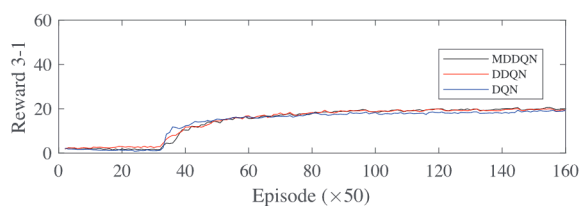


FIGURE 11 Reward for third stage first layer.

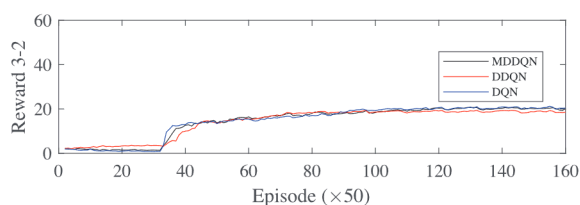


FIGURE 12 Reward for third stage second layer.

From Figure 9, the MDDQN maintains a relatively high reward value at the beginning of training. From the first 50 iterations, it can be found that the reward value of MDDQN is the first to exceed 20. the advantage of MDDQN ratio is more obvious in this stage. From the 50th to the 100th iteration, the reward values of the three algorithms gradually approach each other, and MDDQN still maintains a slight advantage over the other two algorithms. From the 100th iteration onwards, their reward values are very close to each other.

From Figure 10, the reward value of MDDQN is significantly higher than the other two algorithms from the start and is the first to exceed 40. MDDQN maintains a clear advantage over the other two algorithms until the 4000th iteration. Around the 4000th iteration, the three algorithms tend to have close reward values, but MDDQN remains the best. In general, it can be intuitively found that the performance of the proposed MDDQN outperforms DDQN and DQN in the first two stages in the starting phase. As the training progressed, the MDDQN advantage slowly diminished. Overall, in the first two stages MDDQN maintains a better performance relative to the other two algorithms.

As shown in Figures 11 and 12, they are reward values for each of the two layers of third stage. As far as the first layer is concerned it can be seen that the reward values for the three algorithms are low until about 1700 iterations. In the period between 1700 iterations and 2000 iterations, the reward values

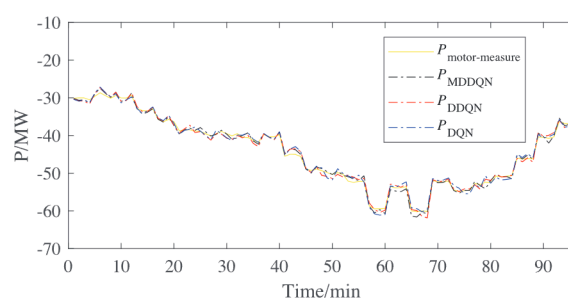


FIGURE 13 Measured and calculated values of motor.

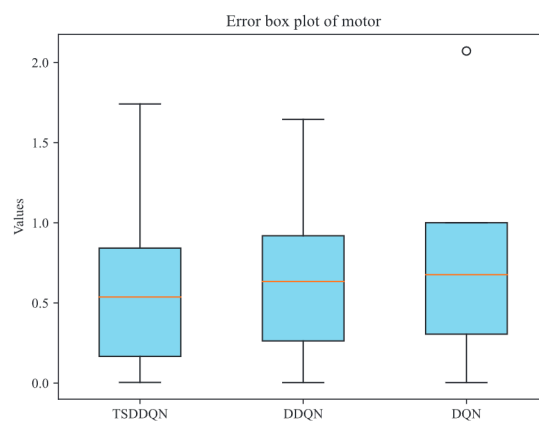


FIGURE 14 Error box plot of motor.

TABLE 4 Error of motor.

Algorithm	Max	Upper-quartile	Median	Lower-quartile	Min
MDDQN	1.741	0.8421	0.5371	0.1667	0.004372
DDQN	1.645	0.9191	0.6340	0.2631	0.003266
DQN	2.072	1.001	0.6764	0.3052	0.003196

increase rapidly reaching close to 20. The reward value profile for the second layer is roughly similar to the first layer. Overall, the performance of the three algorithms does not show a significant difference in the identification of adjustable loads. It can be concluded that all three algorithms can recognize the special load.

As shown in Figure 13, Figure 14 and Table 4, they show the performance of the three algorithms in solving the equivalent induction motor parameters. From Figure 13, all of them are effective in solving the active power of the equivalent induction motor. Figure 14 reflects the error of the three algorithms in solving the equivalent induction motor. Combining Table 4 and Figure 14, it can be found that from the point of view of error, MDDQN maintains the advantage in the upper-quartile, median and lower-quartile.

The results of identifying the total ZIP load are shown in Figure 15. It can be found that the three algorithms can still identify the active power of the equivalent ZIP load.

From Figure 16 and Table 5, the error of the three algorithms in identifying the ZIP loads, MDDQN performs better in the

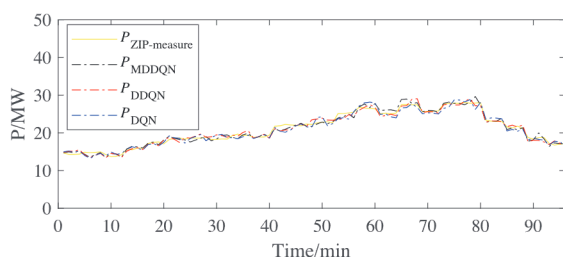


FIGURE 15 Measured and calculated values of ZIP load.

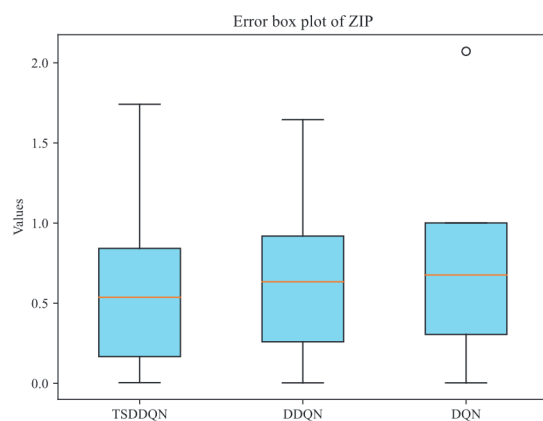


FIGURE 16 Error box plot of ZIP.

TABLE 5 Error of ZIP.

Algorithm	Max	Upper-quartile	Median	Lower-quartile	Min
MDDQN	1.741	0.8421	0.5371	0.1670	0.004372
DDQN	1.645	0.9191	0.6340	0.2593	0.003266
DQN	2.072	1.001	0.6764	0.3052	0.003196

upper quartile, median and lower quartile compared to the other two algorithms.

From Figure 17, all the algorithms are effective in recognizing the active power of EVs.

From Figure 18 and Table 6, compared to the other two algorithms, MDDQN is the smallest in terms of maximum error, upper quartile error and lower quartile error.

As shown in Figure 19, this is the simulation result of identifying the active power of the energy storage device. Around the 95th minute, recognition of DDQN results produced a notice-

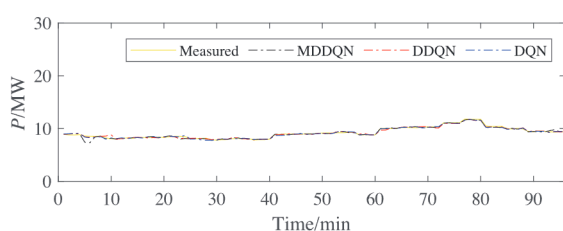


FIGURE 17 Measured and calculated values of EV loads.

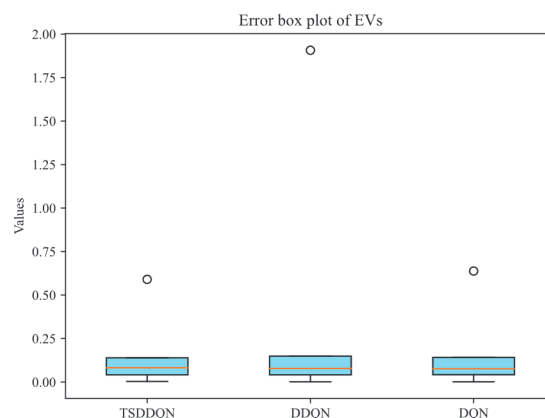


FIGURE 18 Error box plot of EV.

TABLE 6 Error of EVs.

Algorithm	Max	Upper-quartile	Median	Lower-quartile	Min
MDDQN	0.5900	0.1394	0.08200	0.04107	0.003138
DDQN	1.907	0.1488	0.07813	0.04132	0.001129
DQN	0.6379	0.1415	0.07611	0.04183	0.001125

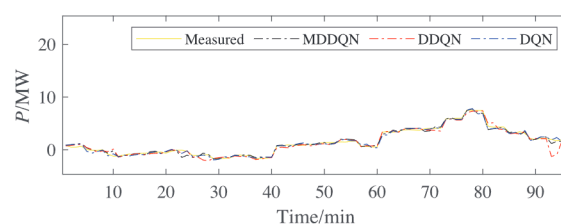


FIGURE 19 Measured and calculated values of storage loads.

able fluctuation. From Figure 20 and Table 7, from the point of view of errors, MDDQN outperforms the other two algorithms in terms of maximum value, upper quartile and lower quartile.

As shown in Figure 21, the three algorithms produced fluctuations in identifying the active power of GZIP around the tenth minute, around the fortieth minute, around the fiftieth to

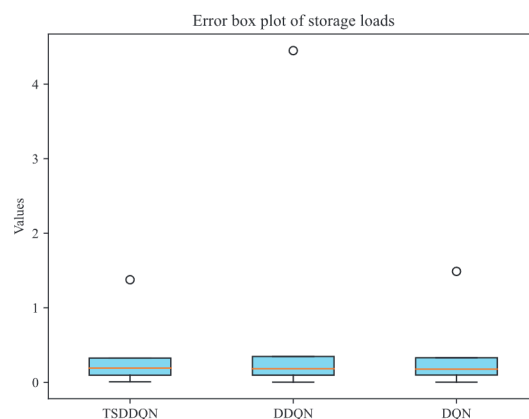
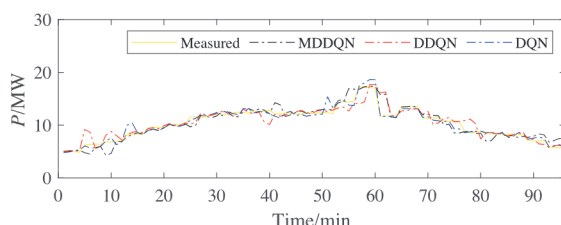
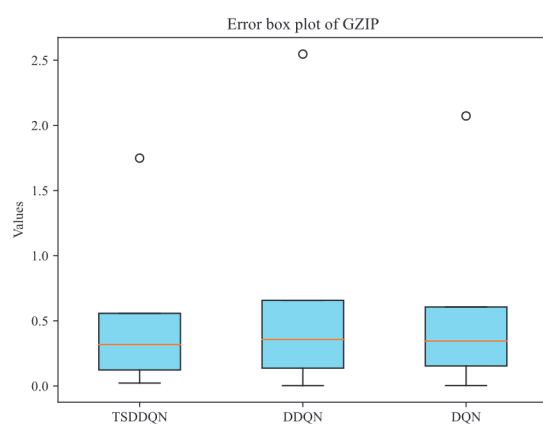


FIGURE 20 Error box plot of storage loads.

TABLE 7 Error of storage loads.

Algorithm	Max	Upper-quartile	Median	Lower-quartile	Min
MDDQN	1.3766	0.3252	0.1913	0.09580	0.007323
DDQN	4.450	0.3471	0.1823	0.09640	0.002634
DQN	1.488	0.3301	0.1776	0.09761	0.002625

**FIGURE 21** Measured and calculated values of GZIP.**FIGURE 22** Error box plot of GZIP.

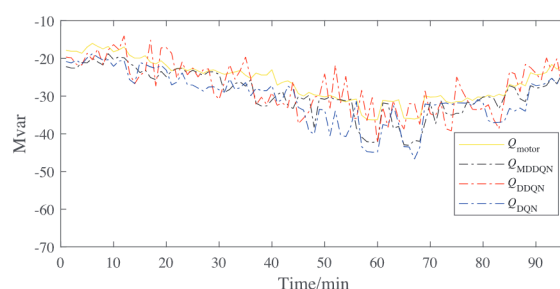
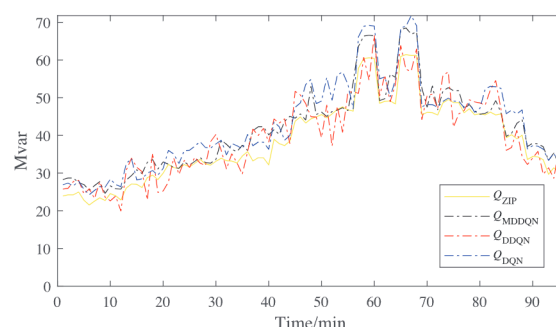
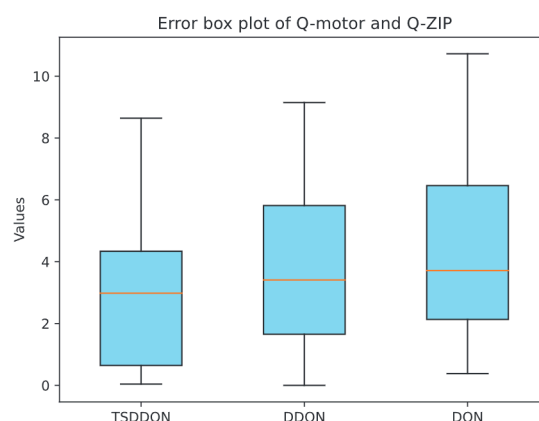
the sixtieth minute, and around the seventieth to the eightieth minute. From Figure 22 and Table 8, the error of MDDQN is minimized in all five key data comparisons.

Figures 23 and 24 demonstrate the results of recognizing reactive power. It can be seen that the volatility of the recognized results for reactive power is stronger as compared to the recognized results for active power. In this case, the performance of MDDQN is optimal among the three algorithms.

From Figure 25 and Table 9, MDDQN maintains its advantage over the other two algorithms for four values: maximum, upper quartile, median, and lower quartile.

TABLE 8 Error of GZIP.

Algorithm	Max	Upper-quartile	Median	Lower-quartile	Min
MDDQN	1.7486	0.5576	0.3182	0.1227	0.002245
DDQN	2.547	0.6572	0.3568	0.1370	0.002800
DQN	2.072	0.6066	0.3447	0.1538	0.003406

**FIGURE 23** Measured and calculated values of Q-motor.**FIGURE 24** Measured and calculated values of Q-ZIP.**FIGURE 25** Error box plot of Q-motor and Q-zip.

In general, from the figures and tables, the methodological framework proposed in this paper is effective in parameter identification for active distribution network equivalence modelling. From the error analysis, MDDQN proposed in this paper is optimal in the three algorithms.

TABLE 9 Error of Q-motor and Q-zip.

Algorithm	Max	Upper-quartile	Median	Lower-quartile	Min
MDDQN	8.642	4.337	2.982	0.6431	0.04143
DDQN	9.148	5.817	3.409	1.654	0.000002536
DQN	10.72	6.461	3.713	2.131	0.3805

6 | CONCLUSION

In this paper, a MDDQN algorithm is proposed for equivalent modelling of ADN to identify the key parameters of the equivalent model. Simulation results show that this method is effective. Eventually, not only the parameters of the equivalent model are obtained, but also the individual identification of adjustable loads is realized. Overall, the method proposed in this paper has a better performance in identifying the components of active power in the equivalent model. In particular, by changing the loss function and improving the adaptivity of the original DDQN algorithm, the MDDQN algorithm proposed in this paper has better performance compared to the other two algorithms. However, the solution error of the proposed method in this paper is relatively large in identifying the equivalent model reactive power. This will be one of the main improvement directions for our future work. In addition, we will also endeavour to improve the performance of this method in different environments in our future work.

We think that load modelling can be widely used in many aspects of power systems. Firstly, in power system control and power system analysis, equivalent models can serve to simplify the topology. Secondly, accurate load modelling can also play an important role in optimal current related studies. Researchers can obtain the required key parameters directly through the equipartition model, which is beneficial to reduce the complexity of researchers' work. Finally, the equipotential model can provide information support for the energy management of the virtual power plant. In conclusion, we believe that the research prospect and application prospect of the isovalue model are very wide.

AUTHOR CONTRIBUTIONS

Jingwen Wang: Methodology; software; writing—original draft. **Jiehui Zheng:** Validation; writing—review and editing. **Zhigang Li:** Formal analysis; visualization. **Qing-Hua Wu:** Supervision.

ACKNOWLEDGEMENTS

This work was supported by the National Natural Science Foundation of China (No. 52477097), State Key Laboratory of Advanced Electromagnetic Technology (Grant No. AET 2024KF005), the GuangDong Basic and Applied Basic Research Foundation (2023B1515250008), and the Young Elite Scientists Sponsorship Program By CSEE (No. CSEE-YESS-2021014).

CONFLICT OF INTEREST STATEMENT

The authors declare no conflicts of interest.

DATA AVAILABILITY STATEMENT

The data are available from the corresponding author on reasonable request.

ORCID

Jiehui Zheng  <https://orcid.org/0000-0003-4928-6524>

Zhigang Li  <https://orcid.org/0000-0003-0464-8800>

REFERENCES

1. Avramidis, I.I., Takis-Defteraios, G.: Flexicurity: Some thoughts about a different smart grid of the future. *IEEE Trans. Smart Grid* 14(2), 1333–1336 (2022)
2. Song, E.Y., FitzPatrick, G.J., Lee, K.B., et al.: A methodology for modeling interoperability of smart sensors in smart grids. *IEEE Trans. Smart Grid* 13(1), 555–563 (2021)
3. Zamani, R., Moghaddam, M.P., Haghifam, M.R.: Evaluating the impact of connectivity on transactive energy in smart grid. *IEEE Trans. Smart Grid* 13(3), 2491–2494 (2021)
4. Shahid, K., Nainar, K., Olsen, R.L., et al.: On the use of common information model for smart grid applications—a conceptual approach. *IEEE Trans. Smart Grid* 12(6), 5060–5072 (2021)
5. Xu, X., Niu, D., Peng, L., et al.: Hierarchical multi-objective optimal planning model of active distribution network considering distributed generation and demand-side response. *Sustainable Energy Technol. Assess.* 53, 102438 (2022)
6. Jabbari, M., Rajabi, A., Ghavidel, S., et al.: From active distribution systems to decentralized microgrids: A review on regulations and planning approaches based on operational factors. *Appl. Energy* 253, 113543 (2019). <https://doi.org/10.1016/j.apenergy.2019.113543>
7. Ren, H., Zhang, A., Wang, F., et al.: Optimal scheduling of an EV aggregator for demand response considering triple level benefits of three-parties. *Int. J. Electr. Power Energy Syst.* 125, 106447 (2021)
8. Park, K., Moon, I.: Multi-agent deep reinforcement learning approach for EV charging scheduling in a smart grid. *Appl. Energy* 328, 120111 (2022)
9. Habibidoost, M., Bathaee, S.M.T.: A self-supporting approach to EV agent participation in smart grid. *Int. J. Electr. Power Energy Syst.* 99, 394–403 (2018)
10. Borji, A., Torres, J.L.R., Popov, M.: Fundamental study on the influence of dynamic load and distributed energy resources on power system short-term voltage stability. *Int. J. Electr. Power Energy Syst.* 131(2), 107141 (2021). <https://doi.org/10.1016/j.ijepes.2021.107141>
11. Bennett, M., Liu, Y.: The impact of large-scale dynamic load modeling on frequency response in the U.S. Eastern Interconnection. *Int. J. Electr. Power Energy Syst.* 120(3), 105983 (2020). <https://doi.org/10.1016/j.ijepes.2020.105983>
12. Zheng, J.H., Xiao, W.T., Wu, C., Li, Z.G., Wang, L., Wu, Q.H.: A gradient descent direction based-cumulants method for probabilistic energy flow analysis of individual-based integrated energy systems. *Energy* 265, 126290:1–126290:14 (2023). <https://doi.org/10.1016/j.energy.2022.126290>
13. Kong, X., Yong, C., Wang, C., et al.: Optimal strategy of active distribution network considering source-network-load. *IET Gener. Transm. Distrib.* 13(1), 5586–5596 (2019). <https://doi.org/10.1049/iet-gtd.2018.5781>
14. Chaspierre, G., Denis, G., Panciatici, P., et al.: A dynamic equivalent of active distribution network: Derivation, update, validation and use cases. *IEEE Open Access J. Power Energy* 8, 497–509 (2021)
15. Baviskar, A., Das, K., Koivisto, M., et al.: Multi-voltage level active distribution network with large share of weather-dependent generation. *IEEE Trans. Power Syst.* 37(6), 4874–4884 (2022)
16. Wang, P., Zhang, Z., Huang, Q., et al.: Robustness-improved method for measurement-based equivalent modeling of active distribution network. *IEEE Trans. Ind. Appl.* PP(99), 1–1 (2021). <https://doi.org/10.1109/TIA.2021.3057358>
17. Fuchs, A., Demiray, T., Larsson, M.: Aggregated models of active distribution networks for stability studies of large transmission systems. *Electr. Power Syst. Res.* 212, 108607 (2022)
18. Arif, A., Wang, Z., Wang, J., et al.: Load modeling—A review. *IEEE Trans. Smart Grid* 9(6), 5986–5999 (2018)
19. Barzegkar-Ntovom, G.A., Kontis, E.O., Papadopoulos, T.A., et al.: Methodology for evaluating equivalent models for the dynamic analysis of power systems. *IEEE Trans. Power Delivery* 37(6), 5059–5070 (2022)
20. Shang, X., Li, Z., Zheng, J., et al.: Equivalent modeling of active distribution network considering the spatial uncertainty of renewable energy resources. *Int. J. Electr. Power Energy Syst.* 112, 83–91 (2019)

21. Mitrentsis, G., Lens, H.: Data-driven dynamic models of active distribution networks using unsupervised learning techniques on field measurements. *IEEE Trans. Smart Grid* 12(4), 2952–2965 (2021)
22. Cui, M., Wang, J., Wang, Y., et al.: Robust time-varying synthesis load modeling in distribution networks considering voltage disturbances. *IEEE Trans. Power Syst.* 34(6), 4438–4450 (2019)
23. Zhao, J., Wang, Z., Wang, J.: Robust time-varying load modeling for conservation voltage reduction assessment. *IEEE Trans. Smart Grid* 9(4), 3304–3312 (2016)
24. McNicholas, S.M., McNicholas, P.D., Ashlock, D.A.: An evolutionary algorithm with crossover and mutation for model-based clustering. *J. Classif.* 38, 264–279 (2021)
25. Regulski, P., Vilchis-Rodriguez, D.S., Djurović, S., et al.: Estimation of composite load model parameters using an improved particle swarm optimization method. *IEEE Trans. Power Delivery* 30(2), 553–560 (2014)
26. Zheng, J.H., Wang, W.H., Li, Z., et al.: Multi-layer double deep Q network for active distribution network equivalent modeling with internal identification for EV loads. *Appl. Soft Comput.* 147, 110834 (2023)
27. Liao, H.L., Wu, Q.H., Jiang, L.: Multi-objective optimization by reinforcement learning for power system dispatch and voltage stability. In: *Proceedings of the 2010 IEEE PES Innovative Smart Grid Technologies Conference Europe (ISGT Europe)*, pp. 1–8. IEEE, Piscataway, NJ (2010)
28. Jiang, C.X., Jing, Z.X., Cui, X.R., et al.: Multiple agents and reinforcement learning for modelling charging loads of electric taxis. *Appl. Energy* 222(JUL.15), 158–168 (2018). <https://doi.org/10.1016/j.apenergy.2018.03.164>
29. Zhu, Z., Lin, K., Jain, A.K., et al.: Transfer learning in deep reinforcement learning: A survey. *IEEE Trans. Pattern Anal. Mach. Intell.* 45(11), 13344–13362 (2023)
30. Cao, Z., Wong, K.C., Lin, C.T.: Weak human preference supervision for deep reinforcement learning. *IEEE Trans. Neural Networks Learn. Syst.* 32(12), 5369–5378 (2021)
31. Li, K., Zhang, T., Wang, R., et al.: Deep reinforcement learning for combinatorial optimization: Covering salesman problems. *IEEE Trans. Cybern.* 52(12), 13142–13155 (2021)
32. Mu, G., Wu, F., Zhang, X., et al.: Analytic mechanism for the cumulative effect of wind power fluctuations from single wind farm to wind farm cluster. *CSEE J. Power Energy Syst.* 8(5), 1290–1301 (2022)
33. Trejo, O., Ortiz-Conde, A.: A simple algorithm for high-accuracy maximum power point calculation of photovoltaic systems. *IEEE J. Photovoltaics* 10(6), 1839–1845 (2020)
34. Yong, P., Botterud, A., Zhang, N., et al.: Capacity value of uninterruptible power supply storage. *IEEE Trans. Power Syst.* 38(2), 1763–1766 (2023)
35. Rizvi, S.M.H., Sajan, K.S., Srivastava, A.K.: Synchrophasor based ZIP parameters tracking using ML with adaptive window and data anomalies. *IEEE Trans. Power Syst.* 37(1), 3–13 (2021)
36. Chaudhari, K., Kandasamy, N.K., Krishnan, A., et al.: Agent-based aggregated behavior modeling for electric vehicle charging load. *IEEE Trans. Ind. Inf.* 15(2), 856–868 (2018)
37. Van Hasselt, H., Guez, A., Silver, D.: Deep reinforcement learning with double q-learning. In: *Proceedings of the 13th AAAI Conference on Artificial Intelligence*, pp. 2094–2100. AAAI Press, Washington, DC (2016)
38. Panda, D.K., Turner, O., Das, S., et al.: Prioritized experience replay based deep distributional reinforcement learning for battery operation in microgrids. *J. Cleaner Prod.* 434, 139947 (2023)
39. Bharath, Y.K.: IEEE 14 Bus System Simulink Model, MATLAB Central File Exchange. <https://www.mathworks.com/matlabcentral/fileexchange/46067-ieee-14-bus-system-simulink-model> (2024). Accessed 31 Oct 2024
40. Baran, M.E., Wu, F.F.: Network reconfiguration in distribution systems for loss reduction and load balancing. *IEEE Trans. Power Delivery* 4(2), 1401–1407 (1989)

How to cite this article: Wang, J., Zheng, J., Li, Z., Wu, Q.-H.: Dynamic equivalent modelling for active distributed network considering adjustable loads charging characteristics. *IET Gener. Transm. Distrib.* 18, 4154–4167 (2024). <https://doi.org/10.1049/gtd2.13344>